



Cancer_Epigenetic 2025

Introduction to bash script (2)

Adapted and Updated from MGMA 2024

Huyha

Nov 23 2025

Email: hagiahuy311@gmail.com

Contents

1. More grep
2. More combining and redirection
3. What is bash script?
+Write your first bash script (Hello world)
+Datastream
4. Function
5. Variables, argument
6. Basic math /Question 1
7. IF Statement (Condition) + Case statement Condition/ Classwork
8. Exit Code
9. Loop
10. Practice day
11. Basic Sed + Basic Awk

Homework

1/ Use this code to down

```
mkdir ./homework
```

```
curl -o ./homework/fa1.txt https://drive.usercontent.google.com/download?id=1RvsIx3\_qGI\_znCAWHDyCHKfhUQxm5U6j
```

```
curl -o ./homework/fa2.txt https://drive.usercontent.google.com/download?id=1znAljOXBLbeq4qt0asce185mzKkPgmxxv
```

or you can down 2 files by click to the drive link above

(If you don't have curl please install it.-(sudo apt install curl))

The files after down will be in the homework Directory use (cd ./homework) or (ls ./homework) to see it

Gilde

+ Use “Grep” to extract the organism name of fa1.txt and fa2.txt

+ Use “Grep” and “wc” to find the nucleotide length of file

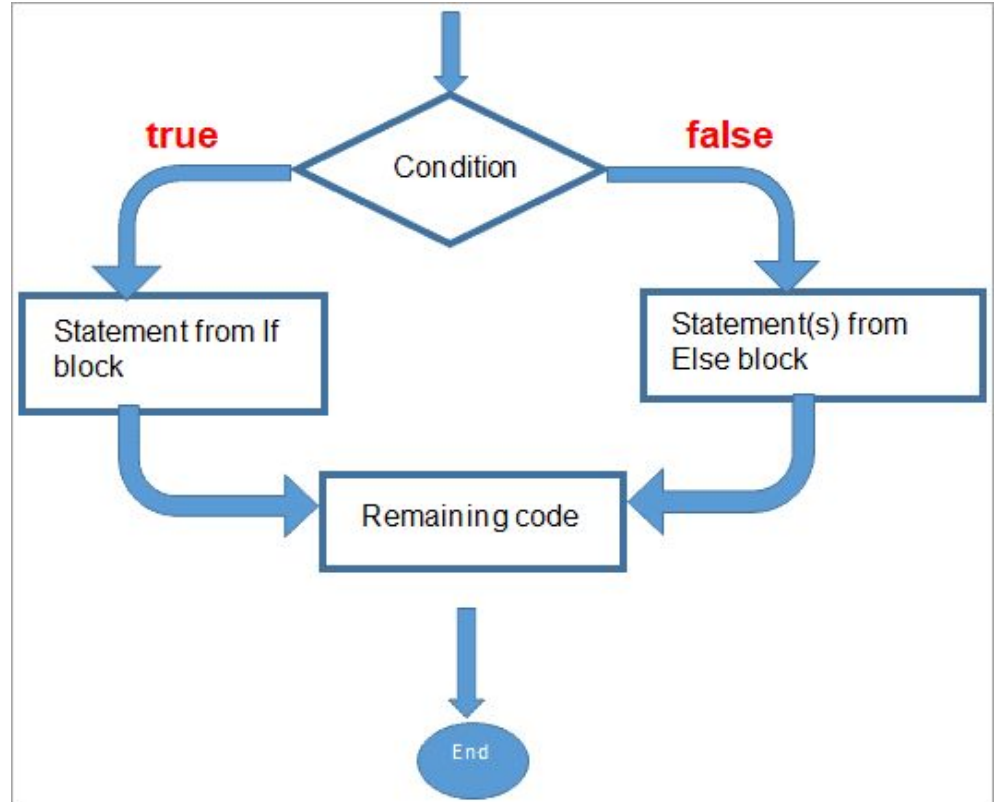
+ Compare the nucleotide length between fa1.txt and fa2.txt

2/ Output the **organism name** and **length** of the bigger nucleotide length file

```
1 >KT719404.1 Porcine circovirus 2 strain PCV2, complete genome
2 ACCAGCGCACTTCGGCAGCGGCAGCACCTCGGCAGCACCTCAGCAGCAACATGCCAGCAAGAAGATGG
3 AAGAAGCGGACCCCAACCCATAAAAGGTGGGTGTTCACTTTGAATAATCCTCCGAAGACGAGCGCAAG
4 AAAATACGGGAGCTTCCAATCTCCCTGTTTGATTATTTTATTGTTGGCGAGGAGGGTAATGAGGAAGGAC
5 GAACACCTCACCTCCAGGGTTCGCTAATTTTGTGAAAAAGCAAACTTTAAATAAAGTGAAGTGGTATCT
6 TGGTGCCCGCTGCCACATCGAGAAAGCCAAAGGAAGTATCAGCAGAAATAAAGATATTGCAGTAAAGAA
7 GGCAACTTACTTATTGAGTGTGGAGCTCCTAGATCTCAAGGACAACGGAGTGACCTGTCTACTGCTGTGA
8 GTACCTTGTGGAGAGCGGGAGTCTGGTGACCGTTGCAGAGCAGCACCCCTGTAACGTTTGTGAGAAATTT
9 CCGCGGGCTGGCTGAACTTTGTAAAGTGAGCGGGAAAATGCAGAAGCGTGATTGGAAGACTAATGTACAC
10 GTCATTGTGGGGCCACCTGGGTGTGGTAAAAGCAAATGGGCTGCTAATTTTGCAGA
11
```

5. If Statement, Case Statement, (Condition)

```
if TEST-COMMAND  
then  
    STATEMENTS  
fi
```



5. If Statement, Case Statement, (Condition)

If syntax

if...else Statement

The Bash `if...else` statement takes the following syntax:

```
if TEST-COMMAND
then
    STATEMENTS1
else
    STATEMENTS2
fi
```

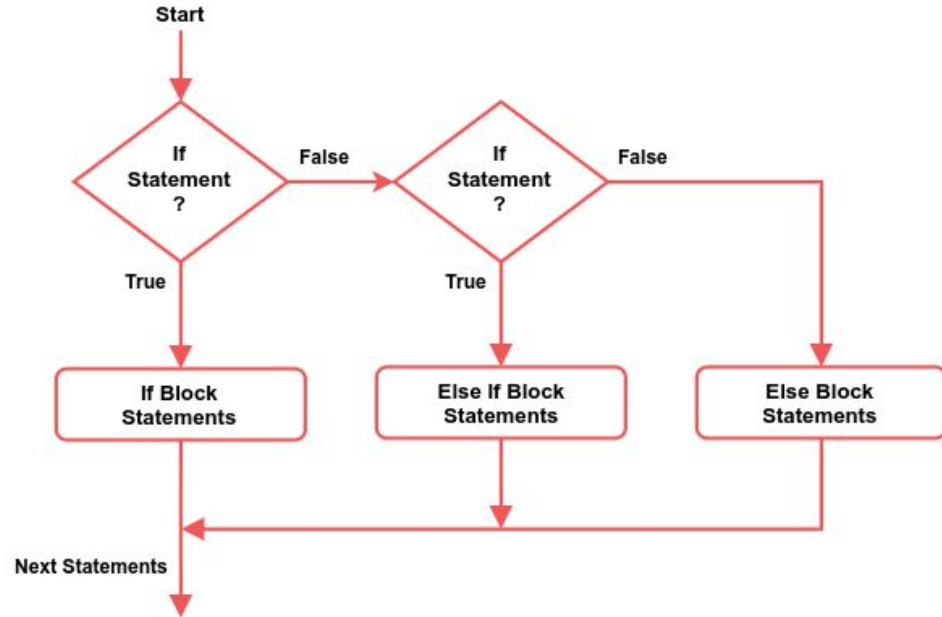
```
baschscript > $ if.sh
1  #!/bin/bash
2
3  echo -n "Enter a number: "
4  read VAR
5
6  if [[ $VAR -gt 10 ]]
7  then
8      echo "The variable is greater than 10."
9  else
10     echo "The variable is equal or less than 10."
11  fi
12
13
```

5. IF statement (continue)

`if...elif...else`

The Bash `if...elif...else`

```
if TEST-COMMAND1
then
    STATEMENTS1
elif TEST-COMMAND2
then
    STATEMENTS2
else
    STATEMENTS3
fi
```



IF...ELSE...IF Statement Flow Diagram

(C) SPEEDYSENSE.COM

5. IF statement (continue)

if...elif...else

The Bash `if...elif...else`

```
if TEST-COMMAND1
then
    STATEMENTS1
elif TEST-COMMAND2
then
    STATEMENTS2
else
    STATEMENTS3
fi
```

```
baschscript > $ if2.sh
1  #!/bin/bash
2
3  echo -n "Enter a number: "
4  read VAR
5
6  if [[ $VAR -gt 10 ]]
7  then
8      echo "The variable is greater than 10."
9  elif [[ $VAR -eq 10 ]]
10 then
11     echo "The variable is equal to 10."
12 else
13     echo "The variable is less than 10."
14 fi
15 |
```


5. NESTED IF

Question:

I have item price

How to find which number
is the largest?

Source:
<https://linuxize.com/post/bash-if-else-statement/>

```
baschscript > $ if3.sh
1  #!/bin/bash
2  $1=VAR1
3  $2=VAR2
4  $3=VAR3
5
6  if [[ $VAR1 -ge $VAR2 ]]
7  then
8      if [[ $VAR1 -ge $VAR3 ]]
9      then
10         echo "$VAR1 is the largest number."
11     else
12         echo "$VAR3 is the largest number."
13     fi
14 else
15     if [[ $VAR2 -ge $VAR3 ]]
16     then
17         echo "$VAR2 is the largest number."
18     else
19         echo "$VAR3 is the largest number."
20     fi
21 fi
22
```

5. Condition

<code>[[-z STRING]]</code>	Empty string
<code>[[-n STRING]]</code>	Not empty string
<code>[[STRING == STRING]]</code>	Equal
<code>[[STRING != STRING]]</code>	Not Equal
<code>[[NUM -eq NUM]]</code>	Equal
<code>[[NUM -ne NUM]]</code>	Not equal
<code>[[NUM -lt NUM]]</code>	Less than
<code>[[NUM -le NUM]]</code>	Less than or equal
<code>[[NUM -gt NUM]]</code>	Greater than
<code>[[NUM -ge NUM]]</code>	Greater than or equal
<code>[[STRING =~ STRING]]</code>	Regex
<code>((NUM < NUM))</code>	Numeric conditions

File conditions

<code>[[-e FILE]]</code>	Exists	<code>[[! EXPR]]</code>	Not
<code>[[-r FILE]]</code>	Readable	<code>[[X && Y]]</code>	And
<code>[[-h FILE]]</code>	Symlink	<code>[[X Y]]</code>	Or
<code>[[-d FILE]]</code>	Directory		
<code>[[-w FILE]]</code>	Writable		
<code>[[-s FILE]]</code>	Size is > 0 bytes		
<code>[[-f FILE]]</code>	File		
<code>[[-x FILE]]</code>	Executable		
<code>[[FILE1 -nt FILE2]]</code>	1 is more recent than 2		
<code>[[FILE1 -ot FILE2]]</code>	2 is more recent than 1		
<code>[[FILE1 -ef FILE2]]</code>	Same files		

Example

Operator	Meaning	Bio-Example
<code>==</code>	Equal to	<code>if ["\$seq" == "ATG"];</code>
<code>!=</code>	Not equal to	<code>if ["\$format" != "fastq"];</code>
<code>-z</code>	String is empty	<code>if [-z "\$sample_name"];</code>
<code>-n</code>	String is not empty	<code>if [-n "\$sample_name"];</code>

```
sample_name=""

# -z checks if the variable has zero length (is empty)
if [ -z "$sample_name" ]; then
    echo "Error: No sample name provided!"
    exit 1
fi
```

Example

Operator	Meaning	Math Symbol
<code>-eq</code>	Equal	<code>=</code>
<code>-ne</code>	Not Equal	<code>!=</code>
<code>-gt</code>	Greater Than	<code>></code>
<code>-lt</code>	Less Than	<code><</code>
<code>-ge</code>	Greater or Equal	<code>>=</code>

```
threads=4
required=8

if [ $threads -lt $required ]; then
    echo "Warning: Only $threads threads available. Assembly will be slow."
else
    echo "Sufficient resources available."
fi
```

Example

Operator	Meaning	Bio-Example
<code>-f</code>	File exists	<code>if [-f "reads.fastq"];</code>
<code>-d</code>	Directory exists	<code>if [-d "output_folder"];</code>
<code>-s</code>	File is not empty (size > 0)	<code>if [-s "assembly.fasta"];</code>
<code>!</code>	NOT (Reverses logic)	<code>if [! -f "file.txt"];</code>

```
input_file="raw_data.fastq"
out_dir="results"

# Check if input exists AND output directory is missing
if [ -f "$input_file" ] && [ ! -d "$out_dir" ]; then
    mkdir "$out_dir"
    echo "Created output directory and ready to start."
elif [ ! -f "$input_file" ]; then
    echo "Critical Error: Input file is missing."
fi
```

Classwork

Input 3 number

+Calculate **Sum, Mean**

+Detect which is the biggest(optional)

+Detect if the biggest (or 3 number input) is **odd** or **even**

Result

```
baschscript > $ practice1.sh
```

```
1  #!/bin/bash
2
3  sum=$(( $1+$2+$3 ))
4  mean=$(( sum/$# ))
5  echo "Sum: $sum"
6  echo "mean $mean"
```

```
7  bg=$1
8  if [[ $2 -gt $bg && $2 -gt $3 ]]
9  then
10     bg=$2
11     echo "$bg is the biggest"
12 elif [[ $3 -gt $bg ]]
13 then
14     bg=$3
15     echo "$bg is the biggest"
16 else
17     echo "$bg is the biggest"
18 fi
19
```

```
19
20 if [[ $(( $bg%2 )) -eq 0 ]];then
21 echo "$bg is even"
22 else
23 echo "$bg is odd"
24 fi
25 |
```

```
./practice1.sh 100 200 303
```

```
Sum: 603
mean 201
303 is the biggest
303 is odd number
```

6. Exit code

```
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ true; echo "$?"  
0  
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ false; echo "$?"  
1
```

In a bash script, an exit code is a numerical value returned by a command or script to indicate its success or failure. Exit codes are useful for scripting as they allow you to determine the outcome of a command or script and take appropriate actions based on that outcome.

Output 0 = TRUE value (success command)

Output 1 = FALSE value (fail command)

\$? = check exit code

```
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ touch /root/test.txt; echo "$?"  
touch: cannot touch '/root/test.txt': Permission denied  
1 (fail command)  
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ touch ~/test.txt; echo "$?"  
0 (success command)
```


Control Exit code

```
1  #!/bin/bash
2
3  touch /root/test.txt
4
5  if [[ $? -eq 0 ]];then
6  |   echo "Done"
7  else
8  |   echo "Error"
9  fi
10 echo "$?"
11
```

```
touch: cannot touch '/root/test.txt': Permission denied
Error
0
```

```
1  #!/bin/bash
2
3  touch /root/test.txt
4
5  if [[ $? -eq 0 ]];then
6  |   echo "Done"
7  |   exit 0
8  else
9  |   echo "Error"
10 |   exit 1
11 fi
12 echo "$?"
13 echo "bla bla bla"
```

```
touch: cannot touch '/root/test.txt': Permission denied
Error
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ echo $?
1
```

Guess question 2

```
3 #1
4 ls > name.txt
5 test -s name.txt
6 echo "1: $?"
7 #2
8 a=005
9 test $a -eq 5
10 echo "2: $?"
11 #3
12 b=1
13 test 001 == $b
14 echo "3: $?"
15 #4
16 test -d /
17 echo "4: $?"
18 #5
19 false && test 2 -gt 1
20 echo 5: $?
```

1. 1,0 2,0 3,1 4,0 5,1

2. 1,0 2,0 3,0 4,1 5,0

3. 1,1 2,0 3,1 4,0 5,1

4. 1,1 2,1 3,0 4,0 5,1

5. 1,0 2,1 3,0 4,1 5,0

Result

```
3 #1
4 ls > name.txt
5 test -s name.txt
6 echo "1: $?"
7 #2
8 a=005
9 test $a -eq 5
10 echo "2: $?"
11 #3
12 b=1
13 test 001 == $b
14 echo "3: $?"
15 #4
16 test -d /
17 echo "4: $?"
18 #5
19 false && test 2 -gt 1
20 echo 5: $?
```

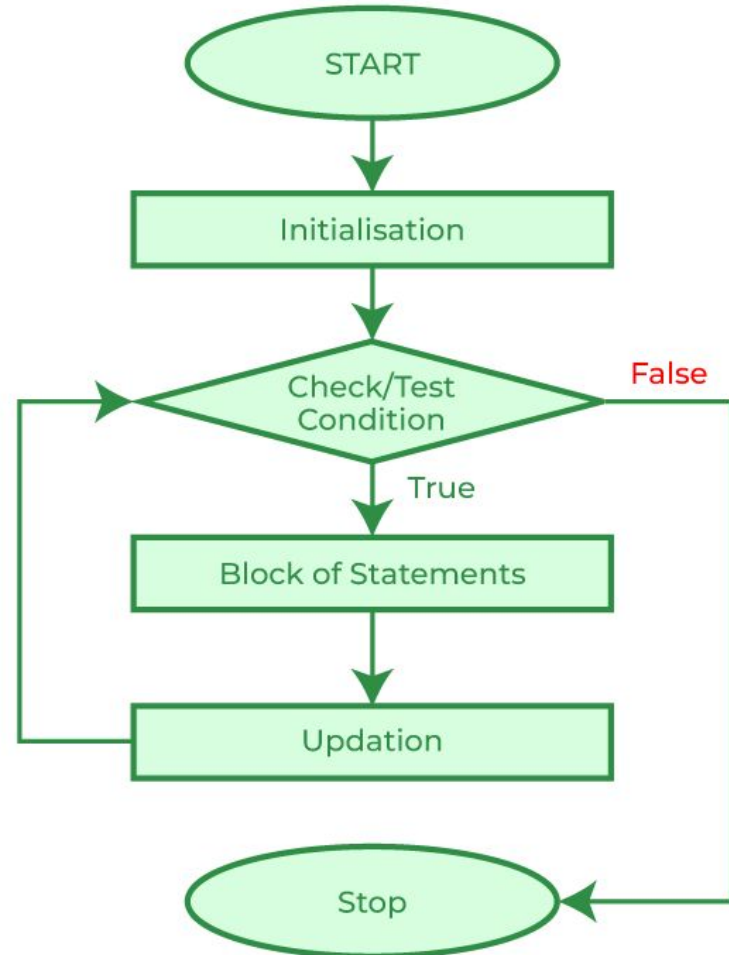
1. 1,0 2,0 3,1 4,0 5,1
2. 1,0 2,0 3,0 4,1 5,0
3. 1,1 2,0 3,1 4,0 5,1
4. 1,1 2,1 3,0 4,0 5,1
5. 1,0 2,1 3,0 4,1 5,0

For loop, While loop

Basic syntax

```
bash
```

```
for item in list
do
    # commands to execute for each item
done
```



Loop structure

For loop, While loop

Loops

Basic for loop

```
for i in /etc/rc.*; do
    echo "$i"
done
```

C-like for loop

```
for ((i = 0 ; i < 100 ; i++)); do
    echo "$i"
done
```

Ranges

```
for i in {1..5}; do
    echo "Welcome $i"
done
```

With step size

```
for i in {5..50..5}; do
    echo "Welcome $i"
done
```

Reading lines

```
while read -r line; do
    echo "$line"
done <file.txt
```

Forever

```
while true; do
    ...
done
```

For loop, while loop

Basic loop

```
baschscript > $ forloop.sh
```

```
1  #!/bin/bash
2  for (( i = 0 ; i <= 2; i++ ))
3  do
4  echo "$i"
5  done
```

```
7  for filename in 7 8 9 huy
8  do
9  echo $filename
10 done
```

```
12 for filename in question*;
13 do
14 echo $filename
15 done
```

```
• (base) huyha@dummysystem:~$ cat forloop.sh
0
1
2
7
8
9
huy
question1.sh
question2.sh
```

For loop, while loop

Example range

```
baschscript > $ forRange.sh
```

```
1  #!/bin/bash
```

```
2
```

```
3  for i in {10..20}
```

```
4  do
```

```
5  echo $i
```

```
6  done
```

```
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ ./forRange.sh
```

```
10
```

```
11
```

```
12
```

```
13
```

```
14
```

```
15
```

```
16
```

```
17
```

```
18
```

```
19
```

```
20
```

For loop, while loop

Example Break, Continue

```
baschscript > $ ForRange1.sh
1  #!/bin/bash
2
3  for i in {10..20}
4  do
5  if [[ $i -eq 12 ]]
6  then
7  echo i touch 12
8  break
9  fi
10 echo $i
11 done
```

```
10
11
i touch 12
```

```
baschscript > $ forRange1.sh
1  #!/bin/bash
2
3  for i in {10..20}
4  do
5  if [[ $i -eq 12 ]]
6  then
7  echo i touch 12
8  continue
9  fi
10 echo $i
11 done
```

```
10
11
i touch 12
13
14
15
16
17
18
19
20
```


Example Continue

```
mkdir result/bowtie2
for i1 in result/fastp/*_1.fq.gz; do
    sample=$(basename "$i1" | cut -d "_" -f1)
    i2=${i1/_1.fq.gz/_2.fq.gz}
    if [[ -s result/bowtie2/nohuman.${sample}.sam ]]; then
        echo "Skipping ${sample}"

        continue
    fi

    micromamba run -n bowtie2 bowtie2 \
        --threads 64 \
        -x /mnt/10T2/huyha/db/bowtie2_db/hg38_index \
        -U result/bbduk/${sample}_1.fq.gz \
        --un-gz result/bowtie2/nohuman.${sample}.fq.gz \
        -S result/bowtie2/nohuman.${sample}.sam &> result/bowtie2/${sample}_bowtie2.log
done
```

For loop, while loop

Reading file

File name.txt

```
baschscript > $ forRead.sh
1  #!/bin/bash
2
3  while read name
4  do
5  echo $name
6  done<name.txt
```

```
baschscript > ≡ name.txt
1  Maya Chen 1112
2  Liam Miller 1114
3  Elena Garcia 1112
4  Noah Jones 3333
5  Sophia Lee 1212
6  Ethan Brown 12312312
7  Olivia Williams 12312312
8  Benjamin Davis 123213
9  Ava Wilson 14125421
10 Michael Taylor 121221
```

Row

Word

```
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ ./forRead.sh
Maya Chen 1112
Liam Miller 1114
Elena Garcia 1112
Noah Jones 3333
Sophia Lee 1212
Ethan Brown 12312312
Olivia Williams 12312312
Benjamin Davis 123213
Ava Wilson 14125421
Michael Taylor 121221
```

```
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ ./forRead1.sh
Maya
Chen
1112
Liam
Miller
1114
Elena
Garcia
1112
Noah

baschscript > $ forRead1.sh
1  #!/bin/bash
2
3  for i in $(cat ./name.txt)
4  do
5  echo $i
6  done
```

Practice 2

Write a script that prints the numbers from 15 to 115 (distance between 2 number is 15). Additionally:

1. **Highlight reaching 60:** When the script encounters the number 60, print a message indicating it reached 60.
2. **Identify odd/even:** For each number, determine if it's odd or even and print that information alongside the number.(optional)
3. **Divisibility by 10:** Indicate if the number is divisible by 10 (a multiple of 10)(optional)

```
15 is odd and indivisible for 10
30 is even and divisible for 10
45 is odd and indivisible for 10
60 is even and divisible for 10
number is reach 60
75 is odd and indivisible for 10
90 is even and divisible for 10
105 is odd and indivisible for 10
```

Result

```
baschscript > $ practice2.sh
1  #!/bin/bash
2
3  for i in {15..115..15}
4  do
5  if [[ $((i%2)) -eq 0 && $((i%10)) -eq 0 ]]
6  then
7  echo " $i is even and divisible for 10"
8  sleep 0.5
9  if [[ $i -eq 60 ]]
10 then
11 echo "number is reach 60"
12 fi
13 else
14 echo " $i is odd and indivisible for 10"
15 fi
16 done
```

Array

1. Bash uses **0-based indexing** (the first item is 0), just like Python. You must use curly braces `{}`.
2. This is the most important syntax you will use. The `@` symbol stands for "All".
3. To count how many samples you have, use the `#` symbol at the start.

```
samples=("Ecoli" "Salmonella" "Listeria")

# Get the first item
echo ${samples[0]}
# Output: Ecoli

# Get the second item
echo ${samples[1]}
# Output: Salmonella
```

```
# Print every item in the list
echo ${samples[@]}
# Output: Ecoli Salmonella Listeria
```

```
# The # counts the elements
echo "Total samples to process: ${#samples[@]}"
# Output: Total samples to process: 3
```

Array



4. If you need to add a sample to an existing list, use `+=`.

5. This is the standard template you will use for 90% of your bioinformatics scripts. It combines the **Array** (Step 3) with the **For Loop**.

`()` = Creating the array.

`[]` = Accessing an index.

`{}` = Expanding the variable

```
samples+=("Staphylococcus")
```

```
#!/bin/bash

# 1. Define the array
genes=("BRCA1" "TP53" "MYC")

# 2. Loop through "All" (@) items
for gene in "${genes[@]}; do

    # 3. Do something with the iterator variable ($gene)
    echo "Searching database for: $gene"
    # grep "$gene" annotation.gff > "${gene}_hits.txt"

done
```

Assignment 1: The Bioinformatics Pipeline

Scenario: You have a list of SRA (Sequence Read Archive) accession numbers. You want to simulate downloading them and checking if they are "paired-end".

Data: Create a script with this array:

```
accessions=("SRR12345" "SRR67890"
"ERR34567")
```

Task:

1. Loop through the `accessions` array.
2. Inside the loop, use `if-else` logic:
 - If the accession starts with "SRR", echo: "Downloading [ID] from NCBI...".
 - Else (if it is ERR), echo: "Downloading [ID] from EBI...".
3. **Bonus:** After the loop finishes, print: "Job Complete. Processed X samples." (Use the array length syntax from Step 4).

```
#!/bin/bash
```

```
# 1. Define the array
```

```
accessions=("SRR12345" "SRR67890" "ERR34567")
```

```
# 2. Loop through the array
```

```
for id in "${accessions[@]}; do
```

```
    # Check if the ID starts with "SRR" using the wildcard *
```

```
    if [[ $id == SRR* ]]; then
```

```
        echo "Downloading $id from NCBI..."
```

```
        # Real command would be: fastq-dump $id
```

```
    else
```

```
        # Assuming if it's not SRR, it's likely ERR (EBI)
```

```
        echo "Downloading $id from EBI..."
```

```
        # Real command would be: wget ftp://ftp.sra.ebi.ac.uk/.../$id
```

```
    fi
```

```
done
```

```
# 3. Bonus: Print the total count
```

```
# We use the # symbol to count the elements
```

```
echo "-----"
```

```
echo "Job Complete. Processed ${#accessions[@]} samples."
```


Cheatsheet

Bash scripting cheatsheet

Example

```
#!/usr/bin/env bash

NAME="John"
echo "Hello $NAME!"
```

Variables

```
NAME="John"
echo $NAME
echo "$NAME"
echo "${NAME}!"
```

String quotes

```
NAME="John"
echo "Hi $NAME" #=> Hi John
echo 'Hi $NAME' #=> Hi $NAME
```

Conditional execution

```
git commit && git push
git commit || echo "Commit failed"
```

Functions

```
get_name() {
    echo "John"
}

echo "You are $(get_name)"
```

See: Functions

Shell execution

```
echo "I'm in $(pwd)"
echo "I'm in `pwd`"
# Same
```

See Command substitution

Conditionals

```
if [ -z "$string" ]; then
    echo "String is empty"
elif [ -n "$string" ]; then
    echo "String is not empty"
fi
```

Brace expansion

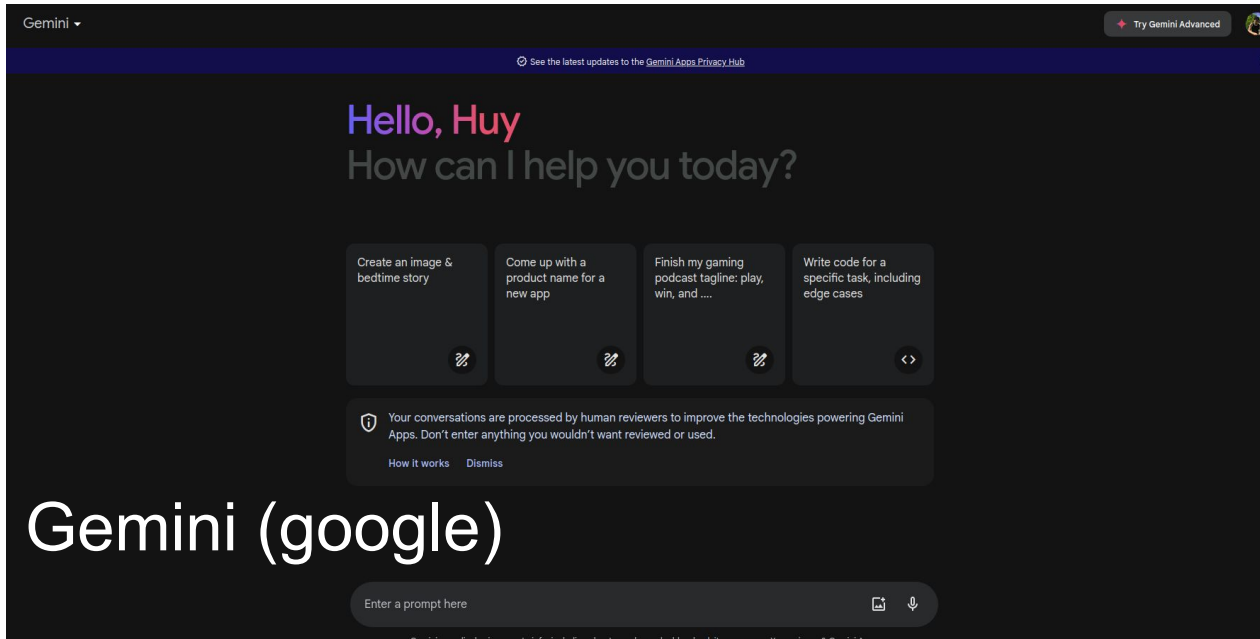
```
echo {A,B}.js
```

Strict mode

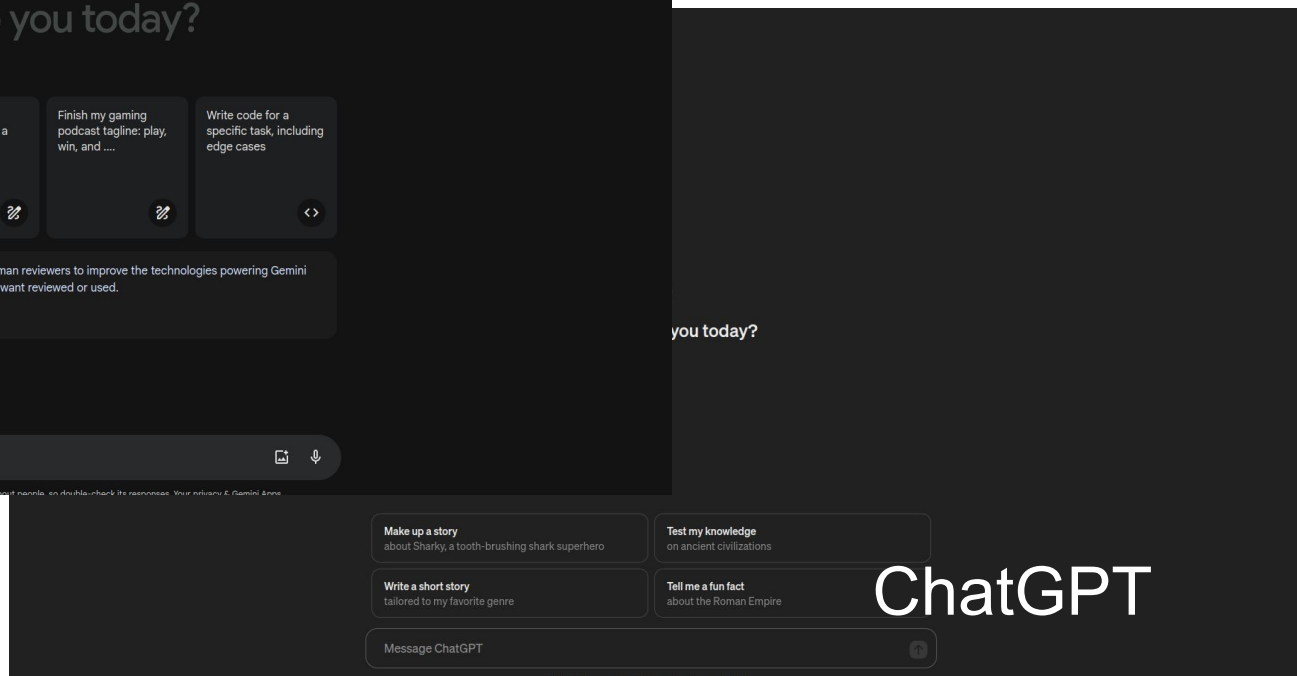
```
set -euo pipefail
IFS=$'\n\t'
```

See: Unofficial bash strict mode

Use AI to learn code



Gemini (google)



ChatGPT

- write bash script to do "Input 3 number
- +Calculate Sum,Mean
- +Detect which is the biggest(optional)
- +Detect if the biggest (or 3 number input) is odd or even

Show drafts ▼



Here's a bash script that prompts for 3 numbers, calculates the sum, mean, finds the biggest number (optional), and determines if it (or any of the inputs) is odd or even:

Bash

```
#!/bin/bash

# Get the three numbers from the user
read -p "Enter first number: " num1
read -p "Enter second number: " num2
read -p "Enter third number: " num3

# Calculate the sum and mean
sum=$((num1 + num2 + num3))
mean=$(bc <<<"scale=2; $sum / 3") # Use bc for floating-point division

# Find the biggest number (optional)
biggest_number=""
find_biggest="yes" # Set to "no" to skip finding biggest

if [[ $find_biggest == "yes" ]]; then
    biggest_number=$(echo -e "$num1\n$num2\n$num3" | sort -nr | head -n 1)
fi

# Check if biggest number (or any input) is odd or even
parity=""
```

```
#!/bin/bash
```

```
# Get the three numbers from the user
read -p "Enter first number: " num1
read -p "Enter second number: " num2
read -p "Enter third number: " num3

# Calculate the sum and mean
sum=$((num1 + num2 + num3))
mean=$(bc <<<"scale=2; $sum / 3") # Use bc for floating-point division

# Find the biggest number (optional)
biggest_number=""
find_biggest="yes" # Set to "no" to skip finding biggest

if [[ $find_biggest == "yes" ]]; then
    biggest_number=$(echo -e "$num1\n$num2\n$num3" | sort -nr | head -n 1)
fi

# Check if biggest number (or any input) is odd or even
parity=""
if [[ $find_biggest == "yes" ]]; then
    if [[ $(($biggest_number % 2)) -eq 0 ]]; then
        parity="even"
    else
        parity="odd"
    fi
else
    # Check each input for odd/even
    for num in $num1 $num2 $num3; do
        if [[ $(($num % 2)) -eq 0 ]]; then
            parity="even (at least one number is even)"
            break # Exit the loop if even is found
        fi
    done
    if [[ -z $parity ]]; then
        parity="odd (all numbers are odd)"
    fi
fi

# Display the results
echo "Sum: $sum"
echo "Mean: $mean"
if [[ $find_biggest == "yes" ]]; then
    echo "Biggest number: $biggest_number"
    echo "Parity of biggest number: $parity"
else
    echo "Parity of inputs: $parity"
fi
```

Homework 1

1. Create file test1.txt and test2 directory in /home/user (~)
2. Write script to find if the files in /home/user is directory or file or other type.

```
• (base) huyha@dummyscomputer:~/huyha/homework/Huyha-Learning/baschscript$ ls ~  
a.out Desktop Documents Downloads homework huyha miniconda3 Music Pictures Public R snap Templates test2 Videos
```

```
• (base) huyha@dummyscomputer:~/huyha/homework/Huyha-Learning/baschscript$ ./homework2.sh  
a.out is file  
Desktop is directory  
Documents is directory  
Downloads is directory  
homework is directory  
huyha is directory  
miniconda3 is directory  
Music is directory  
Pictures is directory  
Public is directory  
R is directory  
snap is directory  
Templates is directory  
test2 is directory  
Videos is directory
```

Summary

1. Loop in bash script
2. Class work
3. Introduce about awk - how to manipulate the text
4. Basic awk
5. Home work